

AI 데이터 분석 · 4일차

# 입력·형변환과 문자열 다루기

값을 받고(input), 바꾸고(형변환), 자르고 찾기(문자열)

input · int·float·str · 인덱싱·슬라이싱 · len·in·count·find

입력 → 변환 → 계산·판단 → 문자열 처리

교안 02\_03(연산자 실습)-02\_04(입력·형변환)-03\_01~03(문자열) 정리

2026-07-16 AI 데이터 분석 교육 정리 (4일차)

학습자 Minho · 국립금오공과대학교 컴퓨터공학부

## 이 책을 읽는 법

어제 배운 변수·자료형·연산자 위에, 오늘은 프로그램이 사용자와 대화하는 법(input)과 데이터의 절반을 차지하는 글자(문자열)를 다룹니다. 파란 세로줄은 파이썬 코드, 초록 ▶ 줄은 실행 결과입니다. 뱃지로 “내장함수 / 연산자 / 문자열 / 메서드 / 개념”을 먼저 구분하고 읽으면 됩니다.

**내장함수** input·int·len   **연산자** += · and · >   **문자열** 생성·인덱싱·슬라이싱   **메서드** count·find   **개념** 원리

### 연결

어제의 마지막 함정 “문자열 w”25w”는 계산이 안 된다”가 오늘 **형변환(int·float·str)**으로 풀립니다. 그리고 그 문자열 자체를 자르고 찾는 법이 오늘의 후반부입니다.

## 목차

|           | 장 / STEP                | 무엇을 배우나                                   |
|-----------|-------------------------|-------------------------------------------|
| CHAPTER 0 | 오늘의 지도 — 받고·바꾸고·다루기     | 연산자 실습 → 입력 → 형변환 → 문자열                   |
| STEP 1    | 연산자, 실습으로 굳히기           | 복합 할당 · 문자열 연산 · 비교 · 논리 · 실전             |
| STEP 2    | 입력받기 — input()          | 사용자 입력 · 안내문구 · 변수 저장 · 항상 str            |
| STEP 3    | 형변환 — int · float · str | 문자열↔숫자 · ValueError · 입력 4단계              |
| STEP 4    | 문자열 만들기                 | 따옴표 · 삼중따옴표 · 이스케이프 · 빈 문자열               |
| STEP 5    | 인덱싱과 슬라이싱               | 0부터 · 음수 · [start:end] · step · 뒤집기       |
| STEP 6    | 문자열 연산과 검색              | 연결 · len · in · count · find · startswith |

## CHAPTER 0

# 오늘의 지도 — 받고·바꾸고·다루기

명령어 하나하나가 어디에 쓰이는지 먼저 그려보기

어제까지는 값을 우리가 코드에 직접 적었습니다(`temp = 85`). 오늘부터는 프로그램이 사용자에게 값을 물어 보고(입력), 그 값을 알맞은 종류로 바꾸고(형변환), 글자 데이터를 자르고 찾습니다(문자열). 큰 흐름은 네 걸음입니다.



앞의 두 걸음(입력·변환)은 “값을 받아 쓸 수 있게 만드는” 준비이고, 뒤의 두 걸음은 어제 배운 연산에 오늘 배울 문자열 처리를 더한 것입니다. 오늘 나오는 도구를 미리 한 표로 봅니다.

| 오늘 나오는 것 | 무엇인가            | 예시                                                                                                    |
|----------|-----------------|-------------------------------------------------------------------------------------------------------|
| 내장함수     | 값을 받고·바꾸고·재는 도구 | <code>input()</code> <code>int()</code> <code>float()</code> <code>str()</code><br><code>len()</code> |
| 연산자      | 계산·판단·갱신 기호     | <code>+=</code> <code>and</code> <code>or</code> <code>not</code> <code>&gt;</code> <code>==</code>   |
| 문자열      | 글자를 만들고 자르는 문법  | <code>"..."</code> <code>s[0]</code> <code>s[1:4]</code> <code>s[::-1]</code>                         |
| 메서드      | 문자열에 붙어 검색·집계   | <code>s.count()</code> <code>s.find()</code>                                                          |

## STEP 1

# 연산자, 실습으로 굳히기

어제 배운 연산을 06\_operator2.py 실습으로 손에 붙이기

어제 산술·비교·논리 연산을 개념으로 배웠으니, 오늘은 실습으로 굳힙니다. 새로 더할 것은 값을 "갱신하는" 복합 할당 연산자와, 문자열 비교의 함정입니다.

**연산자** 복합 할당 += -= \*= /= //= %= 자기 값을 갱신

**정의** 변수의 현재 값에 연산을 하고 그 결과를 다시 같은 변수에 저장하는 줄임 표현. `stock += 50`은 `stock = stock + 50`.

```
stock = 100
stock += 50  # 입고 → 150
stock -= 30  # 출고 → 120
stock += 5   # 반품 → 125
print(stock) # 125
```

▶ 125

**왜 쓰나** 재고·점수·카운터처럼 "기존 값에서 조금씩 바뀌 나가는" 값에 딱입니다. +, -, \*, /, //, % 모두 뒤에 = 를 붙여 쓸 수 있어요.

### 설비 적용

설비 실습(재고 추적): 100 → +=50 → -=30 → +=5 로 입고·출고·반품을 한 변수로 추적합니다.

**연산자** 문자열의 + (연결) · \* (반복) 글자 이어붙이고 반복

**정의** 문자열에서 +는 이어붙이기, \*는 반복입니다. 사이에 공백을 넣으려면 공백 문자열을 더합니다.

```
print("안녕" + "하세요")      # 안녕하세요
print("안녕" + " " + "하세요") # 안녕 하세요
print("=" * 20)               # ===== (구분선)
print("안녕" * 3)             # 안녕안녕안녕
```

▶ 안녕하세요  
▶ 안녕 하세요  
▶ =====  
▶ 안녕안녕안녕

**왜 쓰나** 라벨을 조립하거나("코드: " + code), 보고용 구분선("=" \* 20)을 그릴 때 자주 씁니다. 사이 공백

은 쉼표(,)로 출력하거나 " "를 더해 만듭니다.

**연산자** 비교 연산자와 문자열 비교 == != < > <= >=

**정의** 두 값을 견주어 참·거짓(bool)을 냅니다. 문자열도 비교할 수 있지만, 완전히 같아야 True입니다.

```
print(7 < 16)           # True
print("정상" == "정상") # True
print("Hello" == "hello") # False (대소문자 다름)
print("정상" == "정상 ") # False (뒤 공백 하나 다름)
hi = "안녕"
print(hi == "안녕")      # True (변수 hi의 값과 비교)
```

---

```
▶ True
▶ True
▶ False
▶ False
▶ True
```

**왜 쓰나** 상태 라벨("정상"/"경고")을 판정할 때 씁니다. 대소문자·공백 하나만 달라도 False라는 점이 실무 버그의 단골 원인이예요.

#### 자주 하는 실수

hi == "안녕"에서 hi는 따옴표가 없어 "변수", "안녕"은 따옴표가 있어 "글자"입니다. 만약 "hi" == "안녕"이면 변수가 아니라 글자 "hi"와 비교하는 뜻이 되니 주의.

**연산자** 논리 연산자 and · or · not 참·거짓 역기

**정의** and(둘 다 참), or(하나라도 참), not(뒤집기). 여러 조건을 묶어 복잡한 판단을 만듭니다.

```
print(5 == 5 and 7 == 7) # True
print(5 == 5 or 7 == 5)  # True (하나만 참이어도)
print(not 5 == 5)        # False (True를 뒤집음)
# or는 앞이 True면 뒤는 확인조차 안 함 (short-circuit)
print(5 == 5 or 7 != 7)   # True (뒤는 안 봄)
```

---

```
▶ True
▶ True
▶ False
▶ True
```

**왜 쓰나** "정상 범위 안(하한 그리고 상한)", "위험신호 중 하나라도" 같은 판정에 씁니다. and/or/not 외에 nand-nor-xor도 있지만, 실무 기본은 이 셋입니다.

## 개념

짧은 회로(short-circuit): or는 앞이 True면 뒤를 안 보고, and는 앞이 False면 뒤를 안 봅니다. 뒤에 무거운 계산을 둘 때 속도에 도움이 됩니다.

## 실습 — 정상범위·다중 센서 판정 (실전)

어제·오늘 배운 비교와 논리를 한 번에 씁니다. 온도·압력이 각각 정상 범위인지 비교하고, and로 “둘 다 정상”인지까지 판정합니다. (강사님 강조: 여기서 and를 세 번 쓰게 됩니다.)

```
temp = 85
temp_ok = 60 <= temp and temp <= 90    # 60~90 사이?
print(temp_ok)                          # True
pressure = 5
pres_ok = 3 <= pressure and pressure <= 7
print(pres_ok)                          # True
print(temp_ok and pres_ok)              # True (둘 다 정상)
```

---

- ▶ True
- ▶ True
- ▶ True

## 설비 적용

같은 원리로 **설비 지표**도 계산합니다 — 불량률 =  $\text{defect} / \text{total} * 100$  (23/500 → 4.6%), 가동률 =  $21 / 24 * 100$  (87.5%). 시간 변환은  $500 // 60=8\text{시간}$ ,  $500 \% 60=20\text{분}$ .

## STEP 2

# 입력받기 — input()

프로그램이 사용자에게 값을 물어본다

지금까지 값은 우리가 코드에 직접 적었습니다. `input()`을 쓰면 사용자가 값을 직접 넣습니다. 그래서 하나의 프로그램으로 여러 상황을 처리할 수 있게 됩니다.

**내장함수** `input()` 사용자 입력을 받는 함수

**정의** 실행 중 사용자에게 값을 입력받는 내장 함수. 이 줄을 만나면 프로그램이 멈추고, 사용자가 값을 입력하고 Enter를 누르면 그 다음 코드가 실행됩니다.

```
name = input("이름: ")    # 여기서 멈추고 기다림
print(name)               # 입력한 이름 출력
```

**왜 쓰나** `print`가 "화면에 내보내는" 도구라면, `input`은 "사용자에게서 받아오는" 도구입니다. 방향이 반대예요. 받은 값은 반드시 변수에 저장해야 다시 쓸 수 있습니다(저장 안 하면 그 값은 사라집니다).

### 주의

괄호 안에 안내 문구를 넣으세요: `input("이름: ")`. 안 넣으면 사용자는 무엇을 입력할지 몰라 당황합니다. 끝에 공백 하나를 두면 입력 커서가 붙지 않아 보기 편합니다.

**개념** `input` 결과는 항상 문자열(`str`) 숫자를 입력해도 글자

**정의** `input`으로 받은 값은 무조건 문자열입니다. 사용자가 25를 입력해도 숫자가 아닌 글자 "25"로 저장됩니다.

```
age = input("나이: ")    # 사용자가 25 입력
print(type(age))         # <class 'str'>

a = input("수1: ")       # 3
b = input("수2: ")       # 5
print(a + b)             # 35 (8이 아니라 글자 연결!)

> <class 'str'>
> 35
```

**왜 쓰나** 파이썬은 사용자의 "의도"(숫자인지 글자인지)를 알 수 없어, 안전하게 전부 문자열로 받습니다. 그래서 계산하려면 다음 STEP의 형변환이 반드시 필요합니다.

**내장함수** 여러 값 한 번에 받기 `.split()` 공백으로 나눠 받기

**정의** `input().split()`은 입력을 공백 기준으로 잘라 여러 개로 나눕니다. `map(int, ...)`와 함께 쓰면 숫자 여러 개를 한 줄에 받습니다.

```
name, age = input("이름 나이: ").split() # "홍길동 25" 입력
print(name, age)                        # 홍길동 25

x, y = map(int, input("두 수: ").split()) # "10 4" → 정수로
print(x + y)                            # 14
```

▶ 홍길동 25

▶ 14

**왜 쓰나** 이름·나이, 여러 점수처럼 값을 한 줄에 받을 때 편합니다. `split()`만 쓰면 결과는 여전히 문자열이라, 숫자로 쓰려면 `map(int, ...)` 또는 각각 `int()`로 바꿔야 합니다.



### STEP 3

## 형변환 — int · float · str

받은 글자를 계산할 수 있는 숫자로

input이 항상 문자열을 주기 때문에, 계산하려면 종류를 바꿔야 합니다. 이 "바꾸기"가 형변환입니다. 세 함수 int·float·str만 알면 됩니다.

**내장함수** int() · float() 글자 → 숫자

**정의** int()는 글자로 된 정수를, float()는 글자로 된 실수를 진짜 숫자로 바꿉니다.

```
age = int(input("나이: "))    # "25" → 25
print("내년에는", age + 1, "살")    # 계산 가능!

height = float(input("키: "))    # "173.5" → 173.5
print(type(height))              # <class 'float'>
```

---

▶ 내년에는 26 살  
▶ <class 'float'>

**왜 쓰나** input으로 받은 "25"는 글자라 +1이 안 됩니다. int()로 25로 바꾸면 그때부터 계산됩니다. 나이·개수는 int, 온도·키·평균은 float.

#### 자주 하는 실수

괄호가 두 겹입니다: int(input("나이: ")). 여는 괄호 두 개면 닫는 괄호도 두 개 — 짝을 꼭 확인하세요.

**내장함수** str() 숫자 → 글자

**정의** 숫자를 글자(문자열)로 바꿉니다. 문자열과 숫자를 + 로 이으려면 숫자를 먼저 str()로 바꿔야 합니다.

```
age = 25
print("나이: " + str(age))    # 나이: 25
# print("나이: " + age)      # X TypeError (글자 + 숫자)
```

---

▶ 나이: 25

**왜 쓰나** "온도: 75" 같은 보고 문장을 +로 조립할 때 씁니다. 숫자 25 그대로는 글자와 못 더하니, str(25)로 "25"를 만들어 이어붙입니다. (임표 출력이나 f-string을 쓰면 str() 없이도 됩니다.)

**주의** ValueError – 변환 실패 숫자가 아닌 글자

**정의** 숫자로 바꿀 수 없는 글자를 int()·float()에 넣으면 ValueError가 납니다.

```
int("25")      # 25   (OK)
int("스물다섯") # ValueError (숫자 아님)
int("25.5")     # ValueError (int는 소수점 글자 못 바꿈)
float("25.5")   # 25.5 (이건 OK)
```

**왜 쓰나** 사용자가 엉뚱한 값을 넣으면 여기서 막힙니다. 소수가 있을 수 있으면 int() 대신 float()로 받으세요.  
“숫자 아닌 글자 → 변환 불가”가 ValueError의 핵심 신호입니다.

## 입력 처리 4 단계 — 오늘의 뼈대

input을 쓰는 프로그램은 거의 항상 이 네 박자를 따릅니다. 가장 중요한 건 두 번째, 변환 단계입니다(안 하면 오류).



```
birth = int(input("태어난 해: ")) # 입력 + 변환
age = 2026 - birth                 # 계산
print("나이:", age, "세")         # 출력
```

▶ 태어난 해: 1999 → 나이: 27 세

### 개념

막히면 “지금 어느 단계인가”를 스스로 물어보세요. 대부분의 오류는 **변환 단계를 건너뛰어서** 생깁니다(문자열 + 숫자, 문자열끼리 더해 "35"가 나오는 등).

## STEP 4

# 문자열 만들기

설비 데이터의 절반은 글자다

설비 데이터는 숫자뿐 아니라 센서명·상태 라벨·파일명·코드 같은 글자로 가득합니다. 이 글자 데이터를 “문자열”이라 하고, 오늘부터 본격적으로 다룹니다.

**문자열** 문자열 만들기 – 따옴표 글자를 따옴표로 감싸기

**정의** 글자를 작은따옴표(') 또는 큰따옴표(")로 감싸면 문자열이 됩니다. 시작과 끝의 짝만 맞으면 됩니다.

```
name = '홍길동'
status = "정상"
code = 'EQP-001'
print(name, status, code)  # 홍길동 정상 EQP-001
```

---

▶ 홍길동 정상 EQP-001

**왜 쓰나** 센서명('temp\_sensor\_1'), 상태('NORMAL'), 코드('EQP-001'), 파일명('fd001.csv')이 모두 문자열입니다. 주의: '2025'(문자열)와 2025(숫자)는 종류가 다릅니다 — 전화번호 '010-1234-5678'도 글자예요.

**문자열** 따옴표 안에 따옴표 넣기 안과 밖을 다르게

**정의** 문자열 안에 따옴표를 넣으려면, 안과 밖을 서로 다른 종류의 따옴표로 씁니다.

```
a = "It's normal"      # 안에 ' → 바깥은 "
b = '그는 "정상"'      # 안에 " → 바깥은 '
print(a)
print(b)
```

---

▶ It's normal  
▶ 그는 "정상"

**왜 쓰나** 상태 메시지에 따옴표가 들어갈 때 필요합니다. 안과 밖이 같은 종류면 파이썬이 문자열이 어디서 끝나는지 헷갈려 오류가 납니다.

**문자열** 여러 줄 문자열 – 삼중 따옴표 """ ... """

**정의** 따옴표 세 개(""" 또는 ''')로 감싸면 여러 줄을 줄바꿈 그대로 담을 수 있습니다.

```
notice = """설비 점검 안내
1. 전원 확인
2. 센서 점검"""
print(notice)
```

- ▶ 설비 점검 안내
- ▶ 1. 전원 확인
- ▶ 2. 센서 점검

**왜 쓰나** 점검 안내문·여러 줄 카드처럼 줄바꿈이 있는 긴 글을 그대로 담을 때 편합니다.

**문자열** 이스케이프 문자 `\n` `\t` `\\` 역슬래시로 특수 표현

**정의** 역슬래시(`\`)로 줄바꿈·탭·역슬래시 같은 특수 문자를 표현합니다. 화면엔 두 글자처럼 보여도 동작은 하나입니다.

```
print("1줄\n2줄")      # \n = 줄바꿈
print("이름\t상태")    # \t = 탭(간격)
print("C:\\data")      # \\ = 역슬래시 하나
```

- ▶ 1줄
- ▶ 2줄
- ▶ 이름     상태
- ▶ C:\data

**왜 쓰나** 한 줄짜리 문자열 안에서 줄바꿈·간격을 넣을 때 씁니다. 삼중 따옴표가 “보이는 대로”라면, 이스케이프는 “기호로” 특수문자를 넣는 방법이에요.

#### 개념

빈 문자열 ""(길이 0)과 공백 문자열 " "(공백 한 글자, 길이 1)은 서로 다른 값입니다. `len()`으로 재보면 0과 1로 갈립니다.

## STEP 5

# 인덱싱과 슬라이싱

글자에도 위치 번호가 있다

문자열의 글자 하나하나에는 극장 좌석처럼 자리 번호가 붙어 있습니다. 이 번호로 원하는 글자를 꼭 집거나 (인덱싱), 구간을 잘라냅니다(슬라이싱). 가장 중요한 규칙은 "번호는 0부터"입니다.

## 인덱스 번호표 — "PYTHON"

|         | P  | Y  | T  | H  | O  | N  |
|---------|----|----|----|----|----|----|
| 양수(앞에서) | 0  | 1  | 2  | 3  | 4  | 5  |
| 음수(뒤에서) | -6 | -5 | -4 | -3 | -2 | -1 |

**문자열** 인덱싱 `s[번호]` 글자 하나 꺼내기

**정의** 변수 뒤 대괄호에 번호를 넣어 그 자리 글자 하나를 꺼냅니다. 첫 글자는 0번, 뒤에서는 -1번부터.

```
word = "PYTHON"
print(word[0])  # P (첫 글자, 0번)
print(word[2])  # T (2번)
print(word[-1]) # N (마지막)
print(word[-2]) # O (끝에서 둘째)
```

```
▶ P
▶ T
▶ N
▶ O
```

**왜 쓰나** 설비 코드의 첫 글자로 종류를 구분('P'면 펌프)하는 식으로 씁니다. 글자 수를 몰라도 마지막 글자는 항상 `word[-1]`.

### 자주 하는 실수

없는 번호를 쓰면 `IndexError`. "PYTHON"은 6글자라 마지막은 5번(길이-1)이고, 6번은 없는 자리입니다. "0부터 세기"를 깜빡할 때 자주 납니다.

**문자열** 슬라이싱 `s[start:end]` 구간 잘라내기 (끝 제외)

**정의** 대괄호 안에 시작:끝을 콜론으로 지정해 여러 글자를 잘라냅니다. 시작은 포함, 끝은 제외입니다.

```
word = "PYTHON"
print(word[1:4]) # YTH (1,2,3번 - 4는 제외!)
print(word[0:3]) # PYT
date = "20250115"
print(date[:4]) # 2025 (연)
print(date[4:6]) # 01 (월)
print(date[6:]) # 15 (일)
```

---

- ▶ YTH
- ▶ PYT
- ▶ 2025
- ▶ 01
- ▶ 15

**왜 쓰나** 전화번호 가운데 네 자리, 날짜의 연·월·일처럼 “정해진 자리”를 쪼갤 때 씁니다. 끝 번호가 제외라는 점이 가장 흔한 실수예요 — 가져올 마지막 글자 번호 +1을 end에 씁니다.

#### 개념

생략 규칙: [:n]은 처음부터 n 직전까지, [n:]은 n부터 끝까지, [:]는 전체. 시작·끝을 비우면 “처음/끝”으로 채워집니다.

**문자열** step 과 뒤집기 s[::2] · s[::-1] 건너뛰기·거꾸로

**정의** 대괄호에 콜론을 하나 더 찍어 간격(step)을 줍니다. 음수 step은 뒤에서부터 —[::-1]은 통째로 뒤집기.

```
word = "PYTHON"
print(word[::2]) # PTO (0,2,4번 - 두 칸씩)
print(word[1::2]) # YHN (1번부터 두 칸씩)
print(word[::-1]) # NOHTYP (뒤집기)
```

---

- ▶ PTO
- ▶ YHN
- ▶ NOHTYP

**왜 쓰나** [::-1]은 문자열 뒤집기 공식처럼 통째로 외워두면 유용합니다. 참고로 슬라이싱은 인덱싱과 달리 범위를 넘어가도 오류 없이 “가능한 만큼만” 가져옵니다.

## 인덱싱 vs 슬라이싱 — 한 표로

| 기준    | 인덱싱         | 슬라이싱            |
|-------|-------------|-----------------|
| 대괄호 안 | 번호 하나 [0]   | 시작:끝 [0:3]      |
| 꺼내는 것 | 글자 한 개      | 구간(여러 글자)       |
| 예시    | word[0] → P | word[0:3] → PYT |

| 기준    | 인덱싱                          | 슬라이싱         |
|-------|------------------------------|--------------|
| 범위 초과 | <code>IndexError</code> (엄격) | 오류 없음 (너그러움) |

## 활용 — 코드·날짜를 부분으로 쪼개기

자리 규칙이 정해진 데이터는 인덱싱·슬라이싱으로 필요한 부분만 뽑아 씁니다.

```
code = "PUMP-A-03"
print(code[0])    # P    (종류: 펌프)
print(code[:4])   # PUMP (설비군)

date = "20260716"
print(date[:4], date[4:6], date[6:])    # 2026 07 16
```

---

```
▶ P
▶ PUMP
▶ 2026 07 16
```

### 설비 적용

설비 코드의 첫 글자로 종류를 빠르게 분류하고(`code[0]`), 날짜 문자열을 연·월·일로 쪼개(`date[:4]`) 보고서에 씁니다. 슬라이싱이 데이터 정리의 기본기예요.

## STEP 6

# 문자열 연산과 검색

길이 재고, 들었는지 보고, 위치 찾기

문자열을 만들고 잘랐으니, 이제 “재고 찾습니다.” 길이(len), 포함 여부(in), 개수(count), 위치(find) — 데이터 필터링의 기초가 되는 도구들입니다.

**내장함수** len() · 구분자로 연결 길이 재기 · 라벨 조립

**정의** len()은 문자열의 글자 수(공백 포함)를 알려줍니다. 여러 문자열을 구분자로 이어 라벨을 만들기도 합니다.

```
print(len("PYTHON"))      # 6
print(len("AI 분석"))     # 5 (공백도 한 글자)
print(len(""))           # 0 (빈 문자열)
phone = "01012345678"
print(len(phone))        # 11 (자릿수 검사)
```

---

```
▶ 6
▶ 5
▶ 0
▶ 11
```

**왜 쓰나** 휴대폰 번호 11자리·설비 코드 7자리처럼 “자릿수가 맞는지” 검사할 때 씁니다. 지금은 눈으로 확인하지만, 조건문을 배우면 자동 판단으로 넘어갑니다.

**연산자** in · not in 포함 여부 (참·거짓)

**정의** 특정 단어가 문자열 안에 들어 있는지 참·거짓으로 확인합니다. not in은 그 반대.

```
msg = "설비 고장 발생"
print("고장" in msg)      # True
print("정상" in msg)     # False
print("정상" not in msg)  # True
```

---

```
▶ True
▶ False
▶ True
```

**왜 쓰나** 로그·상태 메시지에서 위험 키워드('WARNING', '고장')를 탐지할 때 씁니다. 결과가 True/False라 다음 단원의 데이터 필터링·조건문으로 바로 이어집니다.

**메서드** .count() · .find() 개수 세기 · 위치 찾기

**정의** count()는 특정 글자·단어가 몇 번 나오는지, find()는 처음 나오는 위치(번호)를 알려줍니다. 못 찾으면



find는 -1.

```
print("banana".count("a"))      # 3
print("a,b,c,d".count(","))     # 3 (항목 4개)
email = "hong@company.com"
at = email.find("@")
print(at)                       # 4
print(email[:at])               # hong (find + 슬라이싱)
print("정상".find("고장"))      # -1 (없음)
```

```
▶ 3
▶ 3
▶ 4
▶ hong
▶ -1
```

**왜 쓰나** 쉽표 개수로 항목 수를 세거나, '@' 위치를 찾아 이메일 아이디만 슬라이싱하는 식으로 씁니다. find는 “못 찾으면 -1”이 신호예요(오류가 아님).

**메서드** .startswith() · .endswith() 시작·끝 검사

**정의** 문자열이 특정 글자로 시작/끝나는지 참·거짓으로 확인합니다.

```
f = "sensor_log.csv"
print(f.startswith("sensor")) # True
print(f.endswith("csv"))     # True
```

```
▶ True
▶ True
```

**왜 쓰나** 파일명이 특정 접두사로 시작하는지, 확장자가 .csv인지 검사할 때 딱입니다. in이 “어디든 포함”이라면, 이 둘은 “정확히 앞/뒤”를 봅니다.

## 부록 A

# 치트시트 — 형변환·문자열 총정리

시험·복습 전에 이 표들만

## A-1. 입력·형변환

| 함수                   | 하는 일               | 예시                              |
|----------------------|--------------------|---------------------------------|
| <code>input()</code> | 사용자 입력 받기 (항상 str) | <code>input("이름: ")</code>      |
| <code>int()</code>   | 글자 → 정수            | <code>int("25") → 25</code>     |
| <code>float()</code> | 글자 → 실수            | <code>float("3.5") → 3.5</code> |
| <code>str()</code>   | 숫자 → 글자            | <code>str(25) → "25"</code>     |

## A-2. 문자열 인덱싱·슬라이싱

| 형태                         | 뜻         | 예시(word="PYTHON")                |
|----------------------------|-----------|----------------------------------|
| <code>s[0]</code>          | 0번 글자 하나  | <code>word[0] → P</code>         |
| <code>s[-1]</code>         | 마지막 글자    | <code>word[-1] → N</code>        |
| <code>s[1:4]</code>        | 구간(끝 제외)  | <code>word[1:4] → YTH</code>     |
| <code>s[:n] / s[n:]</code> | 앞부분 / 뒷부분 | <code>word[:3] → PYT</code>      |
| <code>s[::-1]</code>       | 뒤집기       | <code>word[::-1] → NOHTYP</code> |

## A-3. 문자열 검색 메서드

| 형태                           | 하는 일         | 예시                                   |
|------------------------------|--------------|--------------------------------------|
| <code>len(s)</code>          | 글자 수         | <code>len("AI 분석") → 5</code>        |
| <code>x in s</code>          | 포함 여부(T/F)   | <code>"고장" in msg</code>             |
| <code>s.count(x)</code>      | 개수           | <code>"banana".count("a") → 3</code> |
| <code>s.find(x)</code>       | 첫 위치(없으면 -1) | <code>email.find("@") → 4</code>     |
| <code>s.startswith(x)</code> | ~로 시작?       | <code>f.endswith("csv")</code>       |

## 부록 B

# 오류·합정 사전

오늘 코드에서 걸려 넘어지기 쉬운 곳

오늘 새로 만나는 오류는 대부분 "종류가 안 맞아서" 생깁니다. 왼쪽처럼 하면 오른쪽으로 고치세요.

| X 이렇게 하면                       | 무슨 일                   | ✓ 바로잡기                                       |
|--------------------------------|------------------------|----------------------------------------------|
| <code>"3" + 5</code>           | TypeError (글자+숫자)      | <code>int("3") + 5</code>                    |
| <code>int("정상")</code>         | ValueError (숫자 아님)     | 숫자 글자만 변환                                    |
| <code>int("25.5")</code>       | ValueError (int는 소수 X) | <code>float("25.5")</code>                   |
| <code>word[6]</code>           | IndexError (6은 없음)     | <code>word[5]</code> / <code>word[-1]</code> |
| <code>word[1:4]</code> 가 4 포함? | 아니오 — 끝은 제외            | 가져올 끝+1을 end에                                |
| <code>"Hello"=="hello"</code>  | False (대소문자)           | 철자·공백·대소문자 통일                                |
| <code>abc[len(abc)-1]</code>   | 오류 (str-int)           | <code>abc[len(abc)-1]</code>                 |

### 주의

공통 처방: 계산이 이상하면 `type(값)`으로 종류부터 확인하세요. 특히 input으로 받은 값은 **항상 문자열**이라, 계산 전에 `int()/float()`로 바꿨는지 먼저 의심하면 대부분 풀립니다.

## 부록 C

# 설비 실습 하이라이트 & 다음 시간

오늘 배운 것으로 설비 데이터를 다루다

오늘 배운 입력·형변환·문자열을 실제 설비 소재로 묶어봅니다. 대표 실습을 코드로 정리했습니다.

## 설비 정보 카드 만들기 (형변환 + 문자열 연결)

```
code = "PUMP-A"
state = "정상"
hours = 1200
day = "2026-07-16"
card = "코드: " + code + "\n상태: " + state + "\n가동: " + str(hours) + "시간"
print(card)
```

- ▶ 코드: PUMP-A
- ▶ 상태: 정상
- ▶ 가동: 1200시간

### 설비 적용

여기서 핵심은 숫자 `hours(1200)`를 `str()`로 바꿔야 글자와 +로 이어진다는 점입니다. 날짜 문자열은 `day[:4]`(연) · `day[5:7]`(월)처럼 슬라이싱으로 쪼갤 수 있어요.

## 다음 시간 예고

| 다음 주제       | 한 줄 미리보기                                                                                                            |
|-------------|---------------------------------------------------------------------------------------------------------------------|
| 정리 메서드      | <code>upper/lower</code> (대소문자) · <code>strip</code> (공백 제거) · <code>replace</code> (치환) · <code>split</code> (쪼개기) |
| f-string 심화 | <code>f"{code} / {state}"</code> 로 문장에 값 깔끔하게 끼우기                                                                   |
| 조건문 (if)    | 오늘 만든 참·거짓(in·비교)으로 갈림길 결정 — “고장 포함이면 경고”                                                                           |

### 연결

오늘의 `in`-비교가 만드는 `True/False`, 그리고 `find()`로 찾은 위치가 다음 시간 **조건문·정리 메서드**의 재료가 됩니다. 오늘 배운 게 곧바로 이어져요.

맺음말

## 오늘의 성취와 다음 한 걸음

받고 → 바꾸고 → 다루기, 한 바퀴 완성

오늘 하루로 프로그램이 사용자와 대화하고(input), 받은 글자를 계산 가능한 숫자로 바꾸고(형변환), 문자열을 만들고·자르고·찾는 데까지 왔습니다. 어제까지의 “정해진 값 계산”에서 벗어나, 이제 상황에 따라 달라지는 프로그램을 만들 수 있게 된 것이 오늘의 가장 큰 진전입니다.

### 개념

복습 루틴 추천: ① `07_input.py`를 실행해 값을 넣어보며 “input은 항상 str”을 눈으로 확인 → ② `int()`를 빼고 실행해 일부러 오류(TypeError/ValueError)를 내보고 고치기 → ③ `"PYTHON"`으로 `[0]`, `[-1]`, `[1:4]`, `[::-1]`을 예측한 뒤 실행해 대조 → ④ 설비 코드 문자열에 `in`, `find`, `len`을 써보기. 예측하고 실행해 맞춰보는 습관이 다음 시간 조건문을 쉽게 만듭니다.

— 끝 —